

agent-runtime

Brief: an event-sourced runtime for tool-using LLM agents

Concise, accurate overview for the project owner

Scope of this document

A condensed, accurate summary of the `agent-runtime` project, based only on its actual code, tests and README. For full detail — every module, every event field, every diagram — see the companion `deep-dive.pdf` in the same folder.

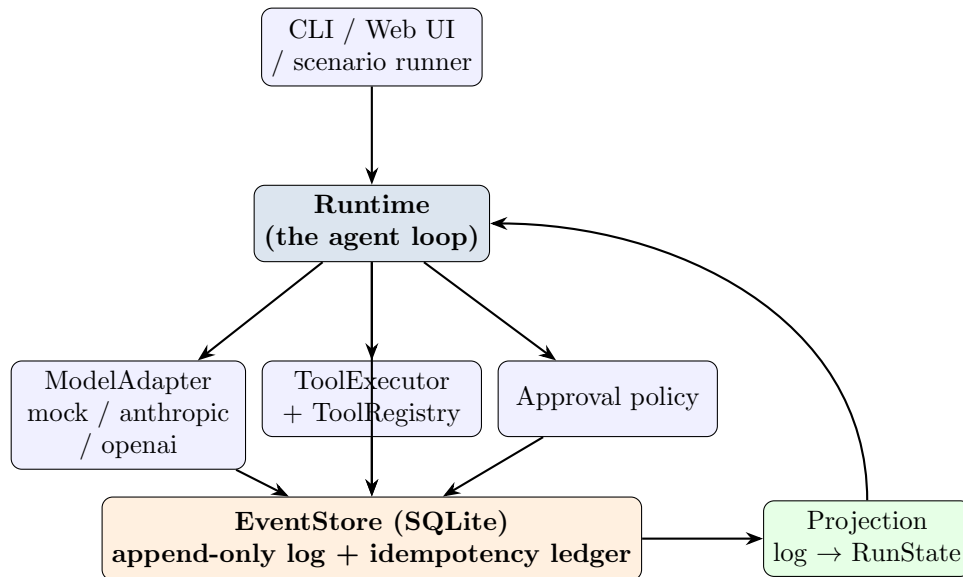
Contents

1	What it is, in one paragraph	2
2	Architecture	2
3	The event log	2
4	The loop	2
5	Idempotency and crash safety	3
6	Deterministic replay	3
7	Failure injection	3
8	Cost accounting	3
9	Tools	3
10	Model adapters	4
11	Agents and scenarios	4
12	CLI and web UI	4
13	Testing	5
14	Honest limitations, in the project's own words	5
15	Glossary	5

1 What it is, in one paragraph

agent-runtime is a Python library and CLI (`agentrun`) for running LLM-driven agents (programs that loop: ask a model what to do, run the tool it requests, feed back the result, repeat) with the operational layer such demos usually skip: every run is recorded as an append-only **event log** in SQLite; the full state of a run is always *computed* from that log, never stored separately. That one decision buys, uniformly: byte-identical **replay** of a past run with no network or tools touched, safe **resume** after a crash with no duplicated side effects, a human **approval gate** before anything with a side effect executes, deliberate **failure injection** to prove the loop degrades cleanly, and a 24-scenario **regression suite** that pinpoints the first event where two model/behavior versions diverge.

2 Architecture



No SQL statement in `store.py` updates or deletes an event row — only inserts and selects. `projection.py` is the single module that turns a log into meaning (`RunState`); the CLI, web UI, replay comparator and scenario assertions all consume it, so they can never disagree about a run’s status.

3 The event log

Table `events(run_id, seq, type, ts, payload)`, primary key `(run_id, seq)`. Twelve event types cover a run’s whole life: `run_created`, `model_requested`, `model_responded`, `model_error`, `tool_requested`, `tool_approved/tool_denied`, `tool_executed/tool_failed`, `checkpoint`, `run_finished/run_failed`. Every event has a *canonical* JSON encoding (sorted keys, no whitespace) so two logs can be compared for exact equality — this is what lets replay claim “byte-identical.”

4 The loop

`Runtime.drive(run_id)` repeats: project the log → if terminal, stop; resolve pending tool calls in order (read-only ones execute; side-effecting ones need an approval decision, or the loop pauses as `waiting_approval`); if nothing pending and under the call limit, ask the model for the next turn, append what happened, loop. Because every step is recomputed from the log, “resume after a crash”

and “continue after a human approves” are the *same* code path as an ordinary iteration — there is no separate recovery logic.

Conversation reconstruction (`build_messages()`) is likewise a pure function of the log: system prompt, request, then one message per subsequent model turn or tool observation. A denial becomes a `{"denied": true, "reason": ...}` observation the model can react to; malformed output becomes a corrective “please reply with tool calls or a final answer” message.

5 Idempotency and crash safety

Side effects must not repeat on resume. The executor derives a key from (`run_id`, `seq` of the `tool_requested` event) — stable across a crash and resume — and writes the result to an `INSERT` OR `IGNORE` ledger *before* the runtime appends `tool_executed`. If the process dies in between (the crash window), a resumed run finds the ledger entry and reports `replayed: true` instead of re-running the handler. `tests/test_resume.py` crashes a run at exactly that instant and asserts a side-effect counter stays at 1.

6 Deterministic replay

`agentrun replay <run-id>` rebuilds a run using four *recorded* stand-ins — a clock, an adapter, a tool executor and an approval policy — each replaying only what the original log recorded, with no network, tool handler or key needed. The real, unmodified runtime then drives the loop, and the two canonical logs are compared; any difference is reported as the exact first diverging event sequence number. `--until <seq>` truncates the source log for stepwise debugging.

7 Failure injection

A YAML-declarable `FaultPlan` names faults at specific points: model faults per adapter call (`malformed`, `adapter.500`) and tool faults per attempt (`timeout`, `exception`). The runtime’s contract: adapter errors retry with exponential backoff then fail the run with a cause; malformed output gets two corrective retries then fails; tool failures surface to the model as ordinary error observations. Nothing escapes `drive()` as an unhandled exception — `tests/test_faults.py` pins this down.

8 Cost accounting

Every `model_responded` event carries token usage and latency. `accounting.py` folds these into a per-run cost using a small, hardcoded price table (the project’s own README flags this table as something that “will drift”). The mock adapter’s usage numbers are fabricated deterministically from message size and always marked `simulated: true` in the CLI and web UI.

9 Tools

Each tool declares a JSON-schema for its arguments, a `side_effect` flag, a timeout and a retry budget (`tools/registry.py`). The executor validates arguments, runs the handler with a timeout, and retries transient failures with backoff while failing fast on expected, deterministic ones (`ToolError`). All file paths are sandboxed to the run’s workspace directory.

Tool	What it does	Side effect
<code>read_file</code>	read a UTF-8 text file	no
<code>write_file</code>	write a UTF-8 text file	yes
<code>list_files</code>	recursively list files	no
<code>http_get</code>	GET an allowlisted URL	no
<code>csv_update</code>	append a row / set a cell in a CSV	yes
<code>draft_email</code>	write a .eml draft file — never sends	yes
<code>calendar_add</code>	insert a row into a per-workspace SQLite calendar	yes
<code>calendar_list</code>	read the calendar back	no

10 Model adapters

One interface, `ModelAdapter.complete(messages, tools) -> ModelTurn`, three implementations: `MockModelAdapter` (a deterministic, offline script player used by tests and scenarios — see Sec. 10), `AnthropicAdapter` (Messages API) and `OpenAIAdapter` (Chat Completions API), both env-key-driven via `httpx`.

Honest scope note (verbatim from the README)

“The Anthropic and OpenAI adapters are implemented against their current APIs and covered by offline contract tests (request building, response parsing, error mapping via injected HTTP transports), but no live API calls were made here because no API keys were available. Everything else, including the full test suite, the CLI and the web UI, runs and was verified entirely offline.”

11 Agents and scenarios

Three example agents, each a fixed system prompt and a restricted tool subset: `ops_assistant` (chases overdue invoices, drafts reminder emails, never sends), `data_entry` (transcribes records into CSV, verifies by reading back), `research_summarizer` (fetches allowlisted pages/notes, writes a sourced summary). 24 YAML scenario files ship under `scenarios/`, 8 per agent, each scripting one or more named *behavior* versions for the mock adapter plus an `expect` block of assertions (exact/contains tool lists, denied tools, final-answer substrings, failure cause, files created, minimum event count, and a structural check that no side effect ever executed before its approval). `agentrun test scenarios/ --behavior v1 --compare v2` runs both versions and reports the first event where their behavior diverges — a regression tripwire for model or prompt changes.

12 CLI and web UI

`agentrun run/resume/replay/runs/approvals/test/serve/agents` cover starting and inspecting runs, replay, the approval queue, the regression suite, and a small FastAPI web UI (`agentrun serve`) that lists runs, shows a run’s timeline, and renders pending approvals as human-readable **diff-style previews** (`web/preview.py`, a real unified diff for file/CSV writes, a plain +-prefixed summary for actions with no natural “before” state) before a human clicks approve or deny.

Known limitation (from the README)

“Approvals via the web UI resume the run synchronously inside the request handler. A slow tool blocks the HTTP worker for its whole timeout. A real deployment needs a worker process consuming a resume queue, with the UI only appending decision events.”

13 Testing

93 test functions across 13 files, all offline — confirmed directly from the files present in `tests/`, matching the README’s own count (“93 tests, all offline”). No test makes a live call to Anthropic or OpenAI.

14 Honest limitations, in the project’s own words

- Tool timeouts abandon a worker thread rather than kill it (a `concurrent.futures` limitation); a genuinely hung tool leaks a thread per attempt.
- Resume repositions the mock adapter’s script/fault counters by counting events; correct for the tested states, but could realign an injected fault sequence off by one in an untested edge case.
- Event payloads carry no schema version — replay is a divergence tripwire today, not a migration story.
- The cost table in `accounting.py` is hardcoded and will drift; the in-house neutral message format ignores streaming and provider-specific parallel tool-call semantics.

15 Glossary

Event sourcing	Storing every state change as an immutable ordered log, and computing current state from it on demand, instead of overwriting stored state.
Idempotent	Doing something twice has the same effect as doing it once.
Replay	Re-driving logic using only recorded data, no live calls, checking the result matches what was recorded.
Side effect	An action with a lasting, observable consequence outside the program (writing a file, sending an email).
Sandbox	A restricted environment limiting what a program can access.
Contract test	A test verifying request/response shape against a real API’s spec without calling the real API.